

CLOUDSHIELD AI

Research Report — Phases 1 to 5

From a broad AI-powered CSPM idea to a focused, testable research direction: DAIL

Prepared for faculty/research review

Status: Phase 1–5 completed; implementation and empirical validation not yet completed.

Date: 24 August 2026

Executive Summary

This report consolidates the research work completed through Phase 5 of the CloudShield AI project. The project began as a broad idea for an AI-assisted cloud security platform that could detect and remediate cloud misconfigurations. Phase 1 established that generic AI-powered Cloud Security Posture Management (CSPM) is already a mature and crowded area, so treating generic AI-CSPM as the main research contribution would be weak. The work was therefore narrowed toward AI-generated Infrastructure-as-Code (IaC), especially Terraform, where the central challenge became the tension between fixing a security problem and preserving infrastructure correctness.

Phase 2 systematically reviewed the literature to test whether the basic LLM → Terraform repair → verification → feedback loop was already solved. The review found strong direct and adjacent prior art: TerraRepair for tool-grounded Terraform repair and dependency context; TerraProbe for layered verification and deceptive-fix detection; SafeFix-Agent for policy/test/sandbox gating; Verifier-First for iterative Terraform validation; and formal-methods systems such as ConVer and PolyVer for contracts/invariants and iterative refinement. This led to the key insight that generic LLM/Terraform repair and ordinary verification loops are already crowded. The remaining research opportunity was a more specific regression-preservation problem: an AI can fix a new issue while breaking a property that was already correct.

That gap became DAIL — Dependency-Aware Invariant Ledger. CloudShield AI remains the overall product concept; DAIL is the narrower research core inside it. DAIL is designed as a stateful, Terraform-specific remediation-control mechanism. It maintains previously verified security and functional properties as protected state, maps infrastructure changes to potentially affected properties using resource/dependency information, re-verifies affected properties, and promotes a candidate Terraform state to trusted status only when the required preservation obligations are satisfied.

Phase 3 formalized the DAIL mechanism: data model, invariant representation, resource identity, dependency impact, invalidation and re-verification, acceptance/promotion, verification engine, feedback loop, baselines, experimental hypotheses, feasibility scope, and threats to validity. Phase 4 then aggressively challenged the contribution against academic, commercial and patent prior art. The main conclusion was not that the individual components are novel — they are not. The contribution candidate is the specific state-promotion and preservation mechanism for iterative LLM-generated Terraform states. Phase 5 then converted that research position into a publication, reproducibility and IP strategy.

At the end of Phase 5, DAIL is therefore not a proven superior technology and not a legally established patent. It is a clearly defined, scoped, falsifiable and experimentally testable research contribution candidate. The next stage is implementation and controlled experimentation, not another broad ideation cycle.

Current Project Status at a Glance

Area	Current conclusion
Overall product	CloudShield AI remains the broader cloud-security product vision.
Research core	DAIL — Dependency-Aware Invariant Ledger.
Research problem	Prevent unintended regressions during iterative LLM-based Terraform remediation.
Phase 1	Problem narrowed; generic AI-CSPM rejected as research contribution.
Phase 2	SLR completed; generic LLM/Terraform repair loop found

	crowded; DAIL gap candidate identified.
Phase 3	DAIL formalized into a testable mechanism and experimental design.
Phase 4	Academic/commercial/patent prior-art challenge completed; contribution refined.
Phase 5	Novelty positioning, patent strategy, experiment mapping, publication and reproducibility strategy completed.
Novelty	Promising but not proven; exact combination not identified in the reviewed evidence.
Patentability	Not established; professional claim-level legal review still required.
Implementation	Not yet completed.
Empirical proof	Not yet available; this is the key task after Phase 5.

1. Overall Research Journey

The project followed a deliberate narrowing process rather than jumping directly into implementation.

1. Start with the broad idea: an AI-assisted cloud security platform that detects and fixes cloud misconfigurations.
2. Perform a reality check against the existing CSPM/CNAPP landscape.
3. Reject generic AI-CSPM as the main research contribution because detection, prioritization, contextualization and AI-assisted remediation are already mature areas.
4. Move to Infrastructure-as-Code, especially Terraform, where reproducible configuration, resource structure and dependencies enable controlled research experiments.
5. Test whether the standard LLM Terraform remediation loop is novel. Phase 2 showed that direct repair and verification systems already exist.
6. Identify the deeper unresolved problem: later AI repairs can regress properties that were previously correct.
7. Define DAIL as the research core that turns previously verified properties into persistent protection obligations for later remediation states.
8. Formalize DAIL, challenge it against prior art, refine the contribution, and design the experiments needed to prove or disprove it.

2. Phase 1 — Problem Discovery, Validation and Narrowing

Phase 1 was not simply an architecture-design phase. Its main role was to determine whether the original problem was significant, whether it was already saturated, what a realistic research problem could be, and what a student team could actually measure and build.

2.1 Original Idea: Broad AI-Powered CSPM

The starting idea was to build CloudShield AI as a broad cloud-security platform that would scan cloud environments, identify security misconfigurations, explain risk, and suggest or automate remediation. In industry language, this is broadly CSPM/CNAPP territory.

The Phase 1 research established that this area is already mature. The project therefore rejected the claim that simply building an AI system that scans cloud security and suggests fixes would constitute a strong research contribution. This was a deliberate rejection, not a failure: it prevented implementation effort from being spent on an already crowded problem.

2.2 Narrowing to IaC and Terraform

The project then moved from generic cloud posture management to Infrastructure-as-Code, especially Terraform. Terraform was selected because it provides a reproducible representation of infrastructure, explicit resource structure, relationships and dependencies, and a convenient basis for controlled before/after experiments.

The research focus became the AI remediation loop: an LLM proposes a Terraform change, security and functional verification are performed, verifier feedback is returned to the LLM, and the process repeats.

2.3 Key Problem Identified in Phase 1

The key concern was not simply whether an AI can remove a security finding. The deeper concern is whether the repair preserves infrastructure correctness. For example, a change that closes a security-group rule may remove the application-to-database connectivity that was previously working. This creates a regression: the new issue may appear fixed while an earlier property has become false.

The Phase 1 material explicitly concluded that claims must remain bounded: formal verification can only establish the selected invariants under the chosen resource semantics; it cannot prove complete real-world cloud security. It also identified the need to avoid assuming that Z3/SMT, full-state verification, or a particular LLM would automatically solve the problem.

2.4 Phase 1 Feasibility and Boundaries

Dimension	Phase 1 conclusion
Problem significance	Strong and measurable.
Generic CSPM novelty	Rejected; too mature/crowded.
Research direction	Promising focus on joint security + functional correctness.
Feasibility	Good with strict scope; controlled AWS/Terraform prototype is realistic.
Measurement	Strong; regressions, repair success, iterations, latency and cost can be measured.
Novelty	Open; must be tested against literature.
Patentability	Open; no conclusion at this stage.
Production readiness	Not claimed.

2.5 Phase 1 Outcome

The major outcome of Phase 1 was a research charter, not a finished system. The project moved from “build an AI cloud security product” toward a measurable research question: how can AI-generated Terraform remediation be made safer when subsequent changes may damage properties that were previously correct?

3. Phase 2 — Systematic Literature Review and Research-Gap Discovery

Phase 2 was the evidence-heavy stage. Its purpose was to test the provisional problem against existing research rather than assume novelty. The review used a structured search/screening/extraction process and explicitly maintained evidence boundaries.

3.1 Search Strategy

The planned search strategy targeted major academic databases and sources, including IEEE Xplore, ACM Digital Library, Web of Science and SpringerLink. Scopus was intended as a primary indexed search source but was inaccessible during the collection stage; this was recorded as a limitation rather than replaced with a fabricated result count.

The workflow also used AI-assisted discovery and citation-expansion tools, including Elicit and Semantic Scholar for discovery/screening support, ResearchRabbit for citation-network expansion, Scite for citation context, Gemini Deep Research for broad prior-art discovery, and Zotero for reference organization. These tools supported the workflow; they did not replace the requirement for evidence verification.

3.2 Screening and Quality Assessment

Papers were screened using INCLUDE / EXCLUDE / MAYBE decisions. Ambiguous studies were deliberately kept in MAYBE status rather than being forced into an unsupported exclusion. High-priority prior-art studies received additional full-text or source verification when available.

The extraction matrix used 21 fields, with particular attention to persistent state, invariant mechanisms, resource identity, dependency-aware invalidation and proof/property caching. Quality scoring used six criteria: objective clarity, methodological rigor, empirical validation, regression/correctness measurement, reproducibility, and threats-to-validity/limitations, each scored 1, 0.5 or 0. The scores were evidence-weighting scores for the material actually available, not immutable judgments of the underlying papers.

Study	Role in Phase 2	Main takeaway for DAIL
TerraRepair	Direct prior art	Tool-grounded Terraform repair and dependency context already exist; persistent cross-iteration invariant preservation was not reported in the reviewed evidence.
TerraProbe	Direct prior art	Layered verification and deceptive-fix detection already exist; it strengthens the oracle/evaluation layer rather than providing persistent repair-state control.
SafeFix-Agent	Direct/low-confidence	Policy/test/sandbox gates show that gated remediation is already studied; persistent invariant memory was not demonstrated in the available evidence.
Verifier-First	Adjacent/directly relevant	Iterative Terraform generation with validate/plan/OPA verifier feedback exists; stateful regression preservation was not demonstrated in the reviewed evidence.
LLMSecConfig	Adjacent	Useful evidence for security/operational preservation, but not the same Terraform-specific DAIL mechanism.
ConVer	Major adjacent threat	LLM-generated contracts/invariants with CEGAR/CEGIS and persistent refinement exist in formal software verification.
PolyVer	Adjacent	Compositional contract/formal refinement exists in C/Rust; domain differs from Terraform.
Zelkova	Foundational	Formal AWS security-policy reasoning exists, but not iterative LLM remediation.
Tiros	Foundational	Formal AWS network-reachability reasoning exists, but not iterative LLM repair control.
UPIR	Adjacent/inspiration	Proof/dependency reuse and incremental verification are relevant

		adjacent ideas; evidence was treated cautiously.
IAMPERE	Provisional/adjacent	Insufficient verified primary evidence in the project set; not used for strong overlap claims.

3.3 The Biggest Phase 2 Discovery

The basic loop “LLM generates/repairs Terraform → verifier checks the result → feedback is returned → LLM tries again” is already increasingly well studied. Therefore, generic LLM + Terraform + verification is not a strong novelty claim. The SLR successfully ruled out that weaker direction before implementation.

The deeper gap candidate was regression-aware, stateful remediation: preserve previously verified properties while the LLM continues to modify infrastructure. The Phase 2 material explicitly described this as a provisional research-gap statement rather than a “first ever” claim.

3.4 DAIL Emerges

DAIL — Dependency-Aware Invariant Ledger — emerged as the leading research candidate. The basic concept was to capture a trusted baseline, receive a security finding, have an LLM generate a constrained patch, verify the new requirement, compare the candidate against protected invariant state, analyze resource/dependency changes, selectively re-verify affected properties, and accept/update the ledger only after verification succeeds. Failed patches remain untrusted and generate feedback for the next repair attempt.

3.5 CloudShield AI vs DAIL

Layer	Purpose
CloudShield AI — Detection/CSPM	Find cloud/IaC security issues.
CloudShield AI — Context/Risk	Explain affected resources, dependencies and severity.
CloudShield AI — AI Remediation	Generate candidate Terraform patches.
DAIL — Research Engine	Maintain invariant state, reason over identity/dependencies, control regression and state promotion.
Verification/Reporting	Provide evidence and show why a candidate was accepted/rejected.

This separation remains central: CloudShield AI is the broader product concept; DAIL is the narrower research mechanism inside it.

3.6 Phase 2 Limitations and Evidence Boundaries

The review was not fully exhaustive. Scopus was inaccessible during collection, not every planned database search had a validated execution log, some screening/extraction decisions relied on abstracts or AI-assisted summaries, publication status varied across peer-reviewed papers/preprints/vendor material, and some named studies had incomplete primary-source evidence. The project deliberately refused to infer persistent invariant ledgers, resource identity enforcement, dependency invalidation or proof caching when the available source did not state them.

Phase 2 also kept several questions open: whether DAIL genuinely reduces regressions, whether full-state verification might be cheaper and simpler, whether selective verification is sound, whether Z3/SMT is necessary, what Terraform subset is feasible, what benchmark to use, and whether the mechanism scales beyond a controlled scope.

3.7 Phase 2 Conclusion

Phase 2 passed. The project was ready for technical contribution design, but DAIL remained provisional until formalization and a new prior-art challenge. The most important success was evidence-based elimination of weak ideas before implementation.

4. Phase 3 — Technical Formalization and DAIL Research Specification

Phase 3 converted the DAIL candidate into a defined, testable research mechanism. The work did not start with coding. Instead, it specified the data model, invariant representation, resource identity, dependency impact, invalidation, verification, acceptance/promotion, feedback loop, baselines, metrics, scope and threats to validity.

4.1 DAIL Core Concept

The core refinement was to stop treating DAIL as merely a database or “ledger.” The more meaningful research object is state promotion: an LLM-generated Terraform candidate state is untrusted until it satisfies both the new remediation requirement and the previously verified protection obligations that the current transition may have affected.

4.2 DAIL Data Model

Entity	Meaning
Resource	A Terraform infrastructure resource and its relevant identity/configuration information.
Dependency	A meaningful relationship between resources that affects property validity or impact.
Invariant	A protected security or functional property that has been verified.
Verification Record	Evidence describing how an invariant was checked and with what result.
State	A trusted infrastructure snapshot/iteration and its associated protected properties.
Patch/Change	An LLM-generated candidate transition from a trusted state to a candidate state.

4.3 Invariant Representation

The recommended conceptual representation is hybrid: human-readable meaning + structured machine representation + machine-verifiable predicate + scope/resources/dependencies + verification evidence + lifecycle status. A security example is “Production EC2 must not allow public SSH.” A functional example is “Application EC2 must retain required connectivity to RDS on TCP 5432.”

DAIL distinguishes protected, affected/needs-recheck, verified/re-established, violated and intentionally retired states. Importantly, invalidation is not the same as failure: it means that prior evidence can no longer be trusted automatically and the property must be re-established.

4.4 Resource Identity and Change Representation

DAIL must distinguish resources that are the same, modified, created, deleted, replaced or uncertain. The system should not rely on raw text diffs to determine infrastructure identity. A conservative rule was adopted: if identity is uncertain, broaden verification rather than silently trust the historical property.

4.5 Dependency-Aware Invalidation

For an invariant I_j , define its relevant dependency scope $Dep(I_j)$. For a change set Δt , the preliminary affected-invariant rule is: an invariant is a candidate for re-verification when its dependency scope intersects the changed resources. This is a candidate algorithmic rule and must be tested for soundness within the supported Terraform subset.

4.6 Formal State Transition

Let S_t be the trusted infrastructure state, P_t an LLM-generated patch, and S'_t the candidate state obtained by applying the patch. DAIL derives a change set, identifies affected invariants, verifies the new requirement, re-establishes affected invariants, performs identity checks and then decides whether S'_t may be promoted. If accepted, S'_t becomes S_{t+1} and the ledger is updated. If rejected, S_t remains trusted and the failure becomes feedback for the next attempt.

4.7 Verification Engine

The recommended verification architecture is hybrid rather than solver-centric: Terraform structural validation and plan/change information, deterministic security invariant checks, functional/reachability checks, and optional SMT/Z3 only where it adds meaningful value. The project explicitly avoids making Z3 the research contribution. Formal verification claims remain bounded by the modeled invariants and resource semantics.

4.8 LLM Feedback and Repair Loop

Rejected patches should return structured, evidence-based feedback: what was successfully fixed, which protected invariant failed, which resource/change caused the failure, and what must be preserved on the next attempt. Failed candidates never become the new trusted baseline. The repair loop is bounded to avoid endless retries and repeated failure.

4.9 Baselines and Experimental Comparison

Three conditions were defined: A — Stateless verification; B — Stateful Full Re-verification; C — DAIL. A tests whether persistent state matters. B tests whether simple full re-verification is enough. C tests whether dependency-aware affected-property re-verification adds value beyond merely storing state and checking everything.

4.10 Scope Lock

The first prototype is deliberately constrained to AWS + Terraform with VPC, EC2, Security Groups and RDS, using a selected set of security and functional invariants. Multi-cloud, all AWS services, multi-account, cross-region, Kubernetes, complete IAM semantics and universal Terraform support are out of scope for the initial research prototype.

4.11 Threats to Validity

The main technical threats are missed dependencies, wrong resource identity, incorrect invariants, verifier errors, deceptive fixes, excessive false rejection, repeated LLM failure, incomplete AWS/Terraform modeling, and the possibility that full re-verification is already sufficient. The project adopts conservative uncertainty handling and explicitly plans to measure missed regressions and false rejection.

4.12 Phase 3 Outcome

Phase 3 moved DAIL from “an interesting idea” to “a defined, testable research mechanism.” The architecture, data model, formal state/transition concepts, candidate algorithm, baselines, hypotheses and scope were all specified. The remaining unknown was whether the mechanism actually produces a measurable benefit.

5. Phase 4 — Deep Prior-Art, Competitor and Patent Validation

Phase 4 deliberately attacked the finalized DAIL specification. The objective was to discover whether the exact mechanism had already been described in academic research, commercial products or patent claims, and to refine the contribution where overlap was found.

5.1 Academic Prior-Art Findings

The academic search confirmed dense surrounding prior art. LLM Terraform repair, layered verification, iterative verifier feedback, contracts/invariants, incremental verification and regression measurement are all established areas. The particularly important recent regression work supports the problem itself rather than destroying DAIL: it shows that iterative LLM-IaC repair can produce security regressions, making preservation a legitimate problem to study.

5.2 Commercial Competitor Findings

Commercial products such as Wiz, Orca Security, Prisma Cloud and Terraform impact-analysis systems cover substantial parts of the broader CloudShield product layer: CSPM, IaC scanning, graph/context, risk analysis, remediation, dependency/change impact and blast-radius analysis. Therefore none of those capabilities should be claimed as DAIL novelty.

The key refinement is that “what might be affected?” is already commercially addressed. DAIL instead proposes to ask: “Which previously verified protection obligations could this transition invalidate, and should the new candidate state be rejected until those obligations are re-established?”

5.3 Patent Findings

The patent landscape is crowded around cloud security posture, IaC remediation, cloud/IaC mapping, resource identity, dependency graphs, drift remediation, deployment gating and LLM-based remediation. The reviewed patent material did not establish a single claim that clearly recites the complete DAIL sequence, but several patents cover major individual components. Therefore broad patent claims around AI cloud remediation or Terraform dependency analysis would be weak, while the state-promotion/preservation interaction is the more promising candidate for professional IP review.

5.4 Exact Overlap Conclusion

The key Phase 4 finding is that the building blocks are not individually novel. The candidate contribution lies in their interaction: a trusted Terraform state carries previously verified security/functional properties as protection obligations; resource/dependency changes determine which obligations are potentially affected; those obligations are re-established; and only then is the candidate state promoted to trusted status.

5.5 Novelty Threat Analysis

The strongest academic threat is that DAIL could be viewed as incremental verification applied to Terraform. This is a legitimate challenge, because change-impact analysis and selective re-verification already exist in formal methods. Another threat is that a simple full-state recheck may provide the same safety benefit at lower complexity. These threats are therefore converted into explicit experiments rather than ignored.

5.6 DAIL Refinement and Final Contribution Lock

After Phase 4, DAIL was refined from “ledger + dependency graph + selective verification” into a stronger concept: a Terraform-specific state-promotion mechanism for iterative LLM remediation. The ledger is treated as protected property state, dependency analysis as impact scope, verification as re-establishment, and the acceptance gate as the central enforcement mechanism.

5.7 Phase 4 Outcome

Phase 4 passed. DAIL survived the academic, commercial and patent challenge with a narrower and more defensible contribution statement, while explicitly acknowledging that novelty and patentability remain provisional until experimental validation and professional legal review.

6. Phase 5 — Novel Contribution, Patentability and Publication Strategy

Phase 5 converted the refined DAIL mechanism into an execution strategy for research, publication and potential IP. It separated the scientific contribution from ordinary implementation, mapped hypotheses to experiments, designed a research-paper structure, established reproducibility requirements and set a cautious IP/publication sequence.

6.1 Final Novelty Positioning

DAIL is not positioned as a new invention of LLM remediation, Terraform repair, dependency analysis, resource identity, invariants or incremental verification. The candidate novelty is the state-promotion semantics: previously verified security and functional properties are carried forward as explicit protection obligations, and their re-establishment becomes a condition for promoting a subsequent AI-generated Terraform state to trusted status.

6.2 Patentability Assessment

Patentability is not established. The surrounding landscape is dense, so the key risks are novelty of individual elements and inventive step/non-obviousness of the combination. The project therefore decided not to rush into filing. A professional patent review should occur after the technical mechanism stabilizes and initial experiments reveal the strongest technical effect. A provisional filing may be a future option depending on legal advice and institutional policy; it is not yet a decision.

6.3 Candidate Patent Claim Architecture

The potential claim architecture centers on: a trusted Terraform state; protected verified-property state; an AI-generated candidate modification; change/resource/dependency analysis; affected-property identification; re-establishment of affected properties; a state-promotion controller; and trusted-state update after successful promotion. Specific implementation choices such as a particular LLM, scanner or solver are not treated as essential to the core concept.

6.4 Research Contribution Decomposition

Layer	What it contains
Core scientific contribution	Stateful, invariant-aware promotion of an AI-generated Terraform state.
Supporting technical mechanisms	Invariant model, resource identity, dependency impact, invalidation/re-establishment, promotion algorithm, conservative uncertainty handling, evidence-backed feedback.
Ordinary implementation	Terraform processing, LLM API integration, storage, graph libraries, scanners/verifiers, optional Z3, UI/API and cloud integration.

6.5 Hypothesis-to-Experiment Mapping

Four headline hypotheses were selected. H1: DAIL reduces unintended regression rate versus stateless verification. H2: DAIL reduces verification work versus stateful full verification without an unacceptable increase in missed regressions. H3: resource-identity-aware preservation reduces restructuring-related regressions. H4: DAIL prevents promotion of candidate states that violate modeled protected invariants. Secondary measures include repair success, false rejection, iteration count, latency, resource churn, protection coverage and verification selectivity.

6.6 Research Paper Strategy

The paper is positioned as an empirical systems/software-engineering and cloud-IaC security paper. The recommended working structure is: Introduction; Background; Related Work; Research Gap and Problem Definition;

DAIL; Implementation; Experimental Methodology; Results; Discussion; Threats to Validity; Limitations; Conclusion. The paper should focus on DAIL and its empirical evaluation rather than on the CloudShield UI or general CSPM product features.

6.7 Publication Strategy

The preferred publication path is a peer-reviewed research conference or focused venue in software engineering, cloud/IaC security, AI-assisted software engineering or related systems research, with a possible expanded journal version later. The final venue should be chosen after results are known. A paper should report the actual experimental result, including negative or neutral results, rather than assuming DAIL wins.

6.8 Reproducibility and Artifact Strategy

Every experimental result should be traceable from paper claim → analysis script → processed result → raw run → Terraform scenario → invariant version → LLM prompt/model configuration → DAIL code version → verification evidence → promotion decision. The repository should keep scenarios, invariants, prompts, implementation, experiments, raw results, processed results, analysis scripts and paper assets versioned.

6.9 IP and Publication Timing

Potentially patent-relevant implementation details should remain controlled until an IP decision is made. The high-level sequence is: stabilize the mechanism → build and test → identify the strongest technical effect → obtain professional IP review → decide on filing → then publish or publicly release sensitive details. Faculty can be briefed internally, but public presentations, open repositories and public papers should be treated as possible disclosure events and reviewed accordingly.

6.10 Phase 5 Final Audit

Phase 5 passed. DAIL is ready to move from research-planning mode into engineering specification, implementation and empirical validation. It is not yet proven superior, novel in the legal sense, or patentable. Those remain questions for the next stage.

7. Overall Conclusions from Phases 1–5

The strongest result of the first five phases is not that a finished product or proven algorithm now exists. The strongest result is that the project has been progressively narrowed using evidence, so that implementation will now be directed at a specific, falsifiable research question instead of an oversized and weakly differentiated cloud-security platform.

7.1 What We Started With

A broad AI-assisted cloud security platform that would detect and fix cloud misconfigurations.

7.2 What We Rejected

Rejected direction	Reason
Generic AI-powered CSPM as the research contribution	Already mature and commercially crowded.
Generic LLM + Terraform + verification loop as the novelty claim	Already increasingly well studied by direct and adjacent research.
Dependency graph as the core novelty	Already established academically and commercially.
Resource identity as the core novelty	Already represented in infrastructure/IaC prior art.
Incremental/selective verification as the core novelty	Established in adjacent formal-verification research.
Z3/SMT as the contribution	A verification technology, not the research contribution itself.

7.3 What We Arrived At

The current research contribution candidate is DAIL: a Terraform-specific, stateful remediation-control mechanism. Its purpose is to make previously verified security and functional properties explicit preservation obligations across iterative LLM-generated changes. A candidate state is not trusted merely because the newest finding disappears; it must also preserve the previously verified properties that the transition may affect.

7.4 The Final Research Logic

The entire project can be summarized as the following chain:

9. Generic CSPM is saturated → reject it as the main research contribution.
10. Terraform/LLM remediation is a better research setting → investigate it systematically.
11. Basic repair-and-verify loops already exist → reject that as the novelty claim.
12. Iterative repair can cause regressions → focus on preservation of earlier correctness.
13. Define protected invariant state + resource/dependency impact + re-establishment + promotion control → DAIL.
14. Challenge DAIL against prior art → refine the contribution to state-promotion semantics.
15. Define baselines and metrics → test whether the mechanism actually helps.

7.5 Final DAIL Working Definition

DAIL (Dependency-Aware Invariant Ledger) is a Terraform-specific, stateful remediation-control mechanism that maintains explicitly verified security and functional properties as protected state, maps infrastructure changes to potentially affected properties through resource and dependency analysis, re-establishes affected properties through verification, and promotes an LLM-generated candidate Terraform state to trusted status only when the required preservation obligations are satisfied.

7.6 What Is Still Unverified

Open question	Why it matters
Will DAIL actually reduce regressions?	Core empirical claim; must be measured.
Will DAIL outperform stateless verification?	Tests whether historical preservation helps.
Will DAIL beat full re-verification?	Tests whether selective dependency-aware control is worth its complexity.
Is selective invalidation sound enough?	A missed affected invariant could create false confidence.
What is the best invariant representation?	Determines reliability and feasibility of verification.
What is the best verification engine?	May be rule-based, reachability-based, SMT-based or hybrid.
What Terraform subset is sufficient?	Must balance semantic richness and student-team feasibility.
Is DAIL legally patentable?	Requires professional claim-level novelty/inventive-step analysis.

8. Readiness for the Next Phase

The project is now ready for Phase 6. Phase 6 should not return to broad product exploration. It should translate the DAIL research specification into an exact Software Requirements Specification (SRS), component interfaces, data schemas, verification contracts, implementation milestones and an experiment harness plan. Only after that should the actual prototype implementation begin.

9. Recommended Phase 6 Starting Point

Phase 6 task	Purpose
6.1 DAIL Software Requirements Specification	Translate the research mechanism into testable software

	requirements.
6.2 Exact internal data schemas	Define state, invariant, dependency, identity, patch and verification records.
6.3 Verification contract	Define exactly what each verifier must accept, reject or mark unknown.
6.4 Implementation architecture	Define modules, interfaces and technology choices.
6.5 Experiment harness specification	Ensure every run is reproducible and auditable from the beginning.
6.6 Unit/integration test strategy	Test DAIL before running the research experiment.
6.7 Prototype plan	Build the smallest implementation capable of testing the hypotheses.

Appendix A — Evidence and Source Basis

This report is based primarily on the authoritative Phase 1 and Phase 2 research artifacts stored in the project files, together with the Phase 3–5 technical design decisions developed from those artifacts during the research workflow. The Phase 1–2 faculty progress report explicitly identifies the authoritative Phase 1 and Phase 2 consolidated reports and the DAIL-specific extraction as the source basis for the faculty-level direction.

Important evidence-boundary note: the Phase 2 SLR was not an exhaustive search across every planned source. Scopus was inaccessible during collection; not every planned database search had a validated execution log; and some papers were evaluated from abstracts or limited source material. The project intentionally kept unsupported overlap claims out of the evidence. These limitations remain part of the research record.

Appendix B — Primary Project Artifacts

Artifact	Role
CloudShield_AI_Phase_1_Consolidated_Research_Report_v1.0.docx	Authoritative Phase 1 problem validation, rejected assumptions, objectives, scope and hypotheses.
CloudShield_AI_Phase_2_Consolidated_SLR_Report_v1.0.docx	Authoritative Phase 2 SLR, thematic synthesis, gap statement, DAIL candidate and limitations.
CloudShield_AI_Phase_1_2_Faculty_Progress_and_Research_Direction_Report.docx	Faculty-level consolidation of the two authoritative phase reports and Phase 3 handoff.
Evidence boundary.md	Evidence-boundary record used to prevent unsupported claims about DAIL overlap.
Stateful, Dependency-Aware Invariant Preservation in LLM-Based Terraform Remediation.pdf	Detailed extraction supporting DAIL-specific comparisons and closest prior art.

Appendix C — Faculty-Level Explanation

30-second version: We started with AI-based cloud security, but generic CSPM was already crowded. We narrowed to AI-generated Terraform security remediation. The literature showed that basic LLM repair-and-verify loops already exist. Our research therefore focuses on the deeper problem that an AI can fix one issue and break another property that was previously correct. DAIL is our mechanism for remembering verified properties and using resource/dependency information to prevent unsafe later state transitions. CloudShield AI remains the overall product.

1-minute version: The product is CloudShield AI, a broader cloud-security platform. The research contribution is DAIL, a Terraform-focused stateful remediation-control mechanism. Instead of treating every AI-generated patch as a fresh state, DAIL maintains protected verified security/functional properties, maps those properties to relevant resources and dependencies, identifies which properties may have been affected by a new change, re-establishes them, and blocks promotion of the candidate state when the preservation conditions fail. The work has survived the literature, competitor and initial patent challenge, but its benefit and patentability still require empirical and professional validation.